


FACULTY OF MATHEMATICS,
PHYSICS AND INFORMATICS
Comenius University
Bratislava

Neural Networks for Computer Vision

Lecture 6: Training Neural Networks I

doc. Ing. Viktor Kocur, PhD., RNDr. Zuzana Černeková, PhD.

22.10.2026

Acknowledgment

The majority of slides are directly adopted from slides for CS231n[1] course at Stanford University!

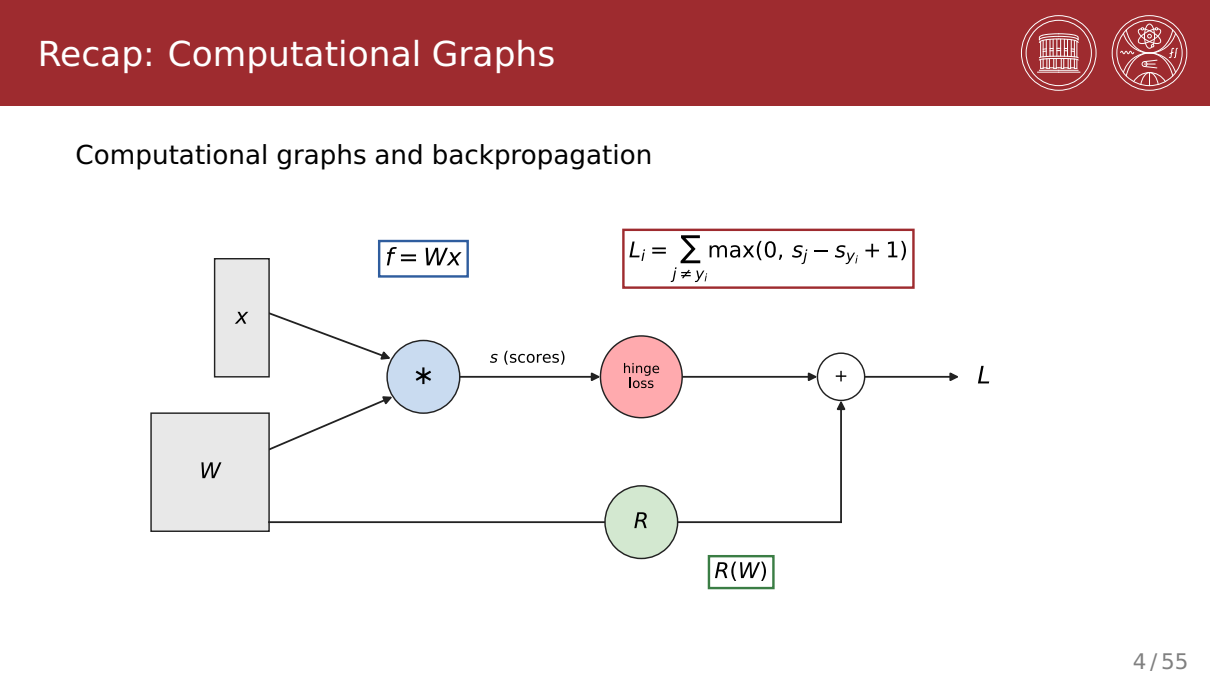
[1] Fei-Fei Li, Ranjay Krishna, and Danfei Xu. *Stanford CS231n lecture slides.* `http://cs231n.stanford.edu/slides/`

Contents



- Activation functions
- Data preprocessing
- Weight initialization
- Transfer learning and self-supervised pretraining
- Batch normalization
  - during training
  - during testing
- Layer normalization and other normalization layers

Recap: Computational Graphs



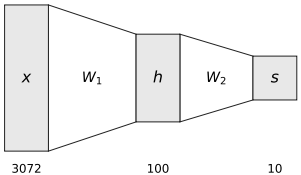
Computational graphs and backpropagation

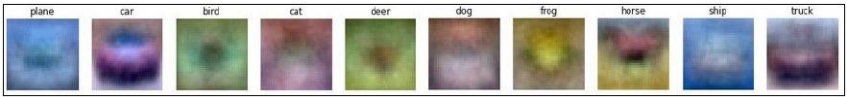
Recap: Neural Networks

- **Before:** Linear score function
- **Now:** 2-layer neural network

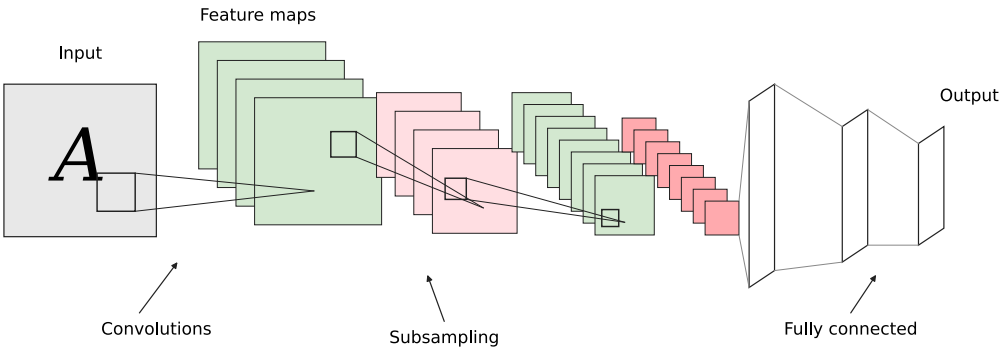
$$f = Wx$$

$$f = W_2 \max(0, W_1 x)$$

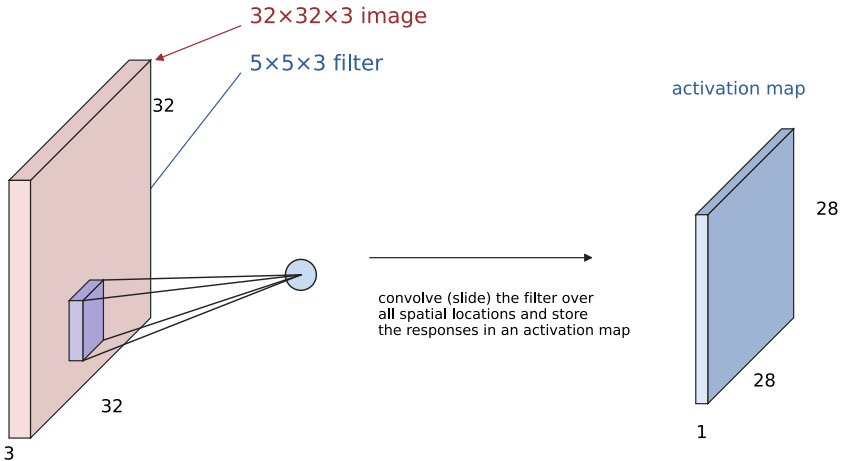




Recap: Convolutional Neural Networks

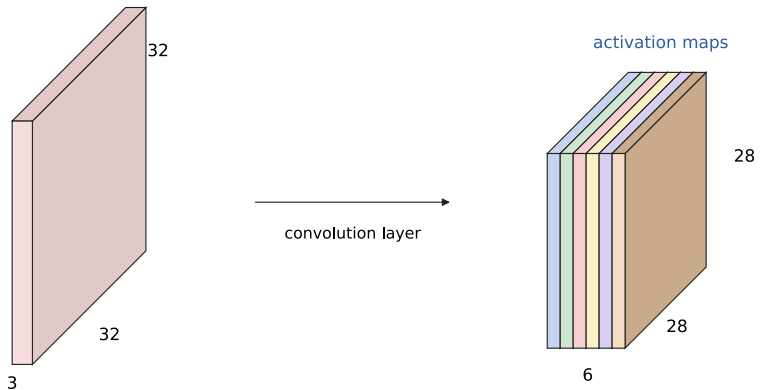


Recap: Convolutional Layer



Recap: Convolutional Layer

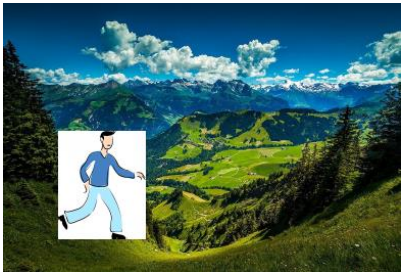
If we have 6 filters of size $5 \times 5 \times 3$, we get 6 separate activation maps:

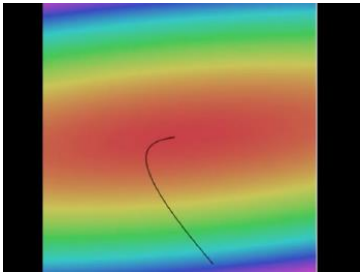


We stack these up to get a “new image” of size $28 \times 28 \times 6$!

Recap: Optimization

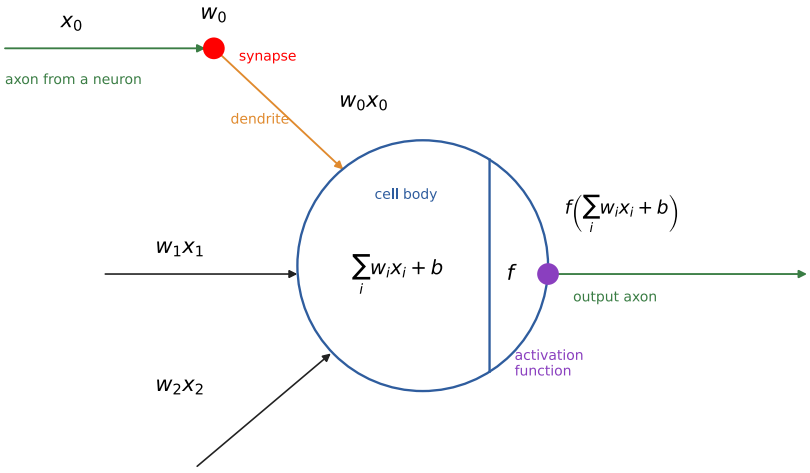
Learn the network parameters through optimization





```
# Vanilla Gradient Descent
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

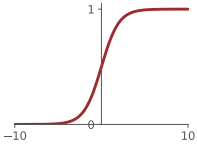
Activation Functions



Activation Functions

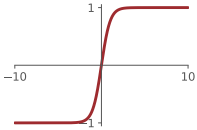
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



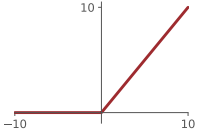
tanh

$\tanh(x)$



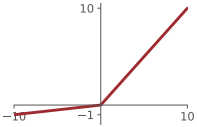
ReLU

$\max(0, x)$



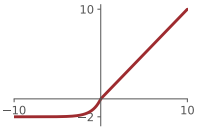
Leaky ReLU

$\max(0.1x, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



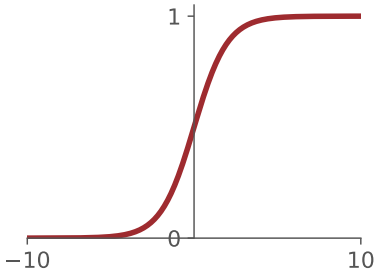
Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

Activation Functions: Sigmoid

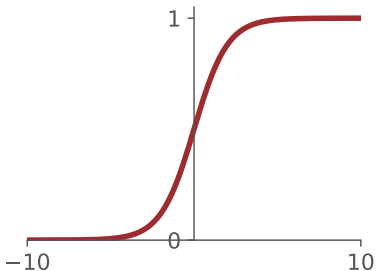
- **Sigmoid** squashes numbers to range $[0, 1]$
- Historically popular since it has a nice interpretation as a saturating "firing rate" of a neuron

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Activation Functions: Sigmoid

- **Sigmoid** squashes numbers to range $[0, 1]$
- Historically popular since it has a nice interpretation as a saturating "firing rate" of a neuron

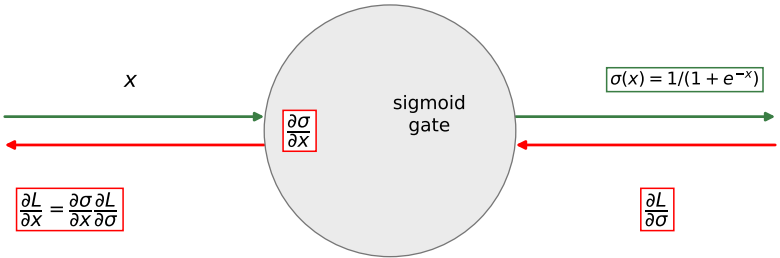


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

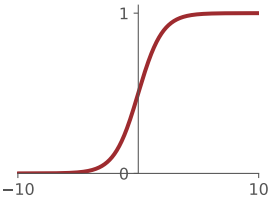
Three problems:

1. Saturated neurons "kill" the gradients

Activation Functions: Sigmoid

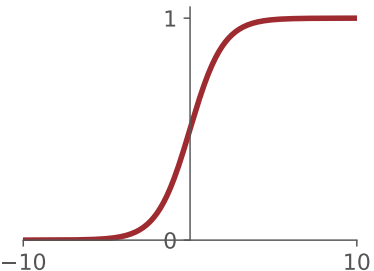


- What happens when $x = -10$?
- What happens when $x = 0$?
- What happens when $x = 10$?



Activation Functions: Sigmoid

- **Sigmoid** squashes numbers to range $[0, 1]$
- Historically popular since it has a nice interpretation as a saturating "firing rate" of a neuron



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Three problems:

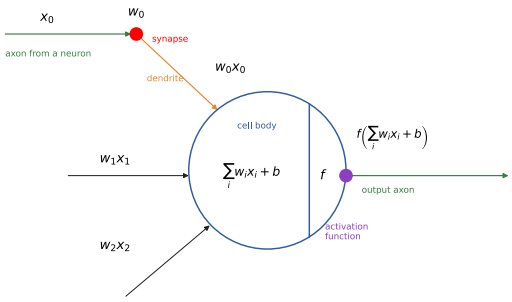
1. Saturated neurons "kill" the gradients
2. Sigmoid outputs are not zero-centered

Activation Functions: Sigmoid

- Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

- What can we say about the gradients on W?

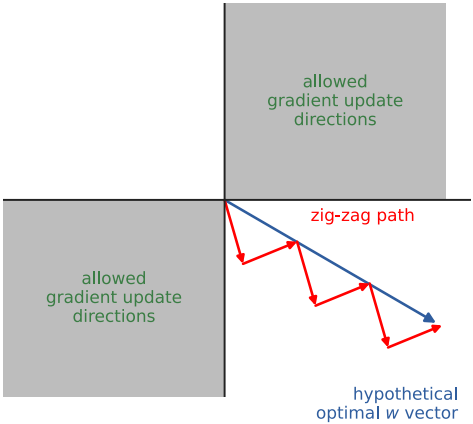


Activation Functions: Sigmoid

- Consider what happens when the input to a neuron is always positive...

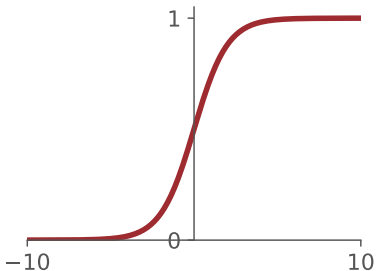
$$f\left(\sum_i w_i x_i + b\right)$$

- What can we say about the gradients on W?
- They are always all positive or all negative :(



Activation Functions: Sigmoid

- **Sigmoid** squashes numbers to range $[0, 1]$
- Historically popular since it has a nice interpretation as a saturating "firing rate" of a neuron

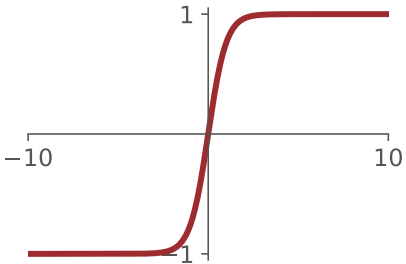


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Three problems:

1. Saturated neurons "kill" the gradients
2. Sigmoid outputs are not zero-centered
3. exp() is a bit computationally expensive

Activation Functions: tanh

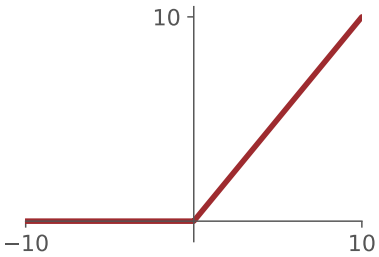


$\tanh(x)$

- **Hyperbolic tangent**
- Squashes numbers to range $[-1, 1]$
- Zero-centered (nice)
- Still kills gradients when saturated :(

Activation Functions: ReLU

Rectified Linear Unit (ReLU)

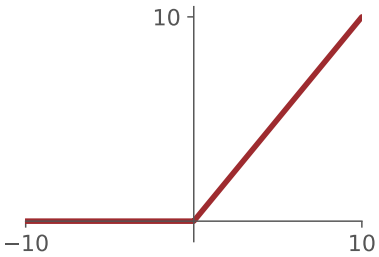


- Computes $f(x) = \max(0, x)$
- Does not saturate (in the $+$ region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)[2]

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks." In: *NeurIPS*. 2012

Activation Functions: ReLU

Rectified Linear Unit (ReLU)

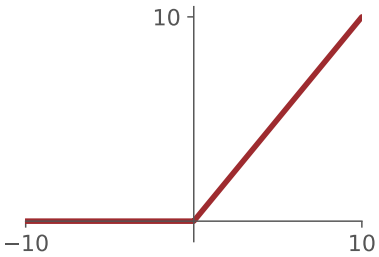


- Computes $f(x) = \max(0, x)$
- Does not saturate (in the $+$ region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)[2]
- Not zero-centered output

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks." In: *NeurIPS*. 2012

Activation Functions: ReLU

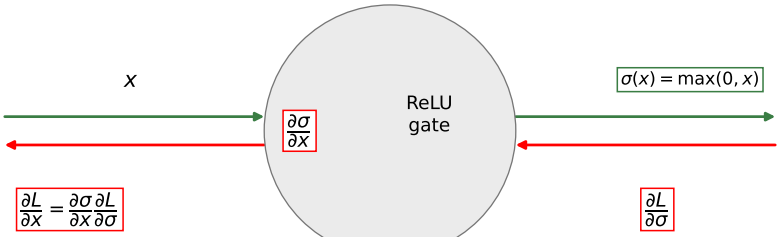
Rectified Linear Unit (ReLU)



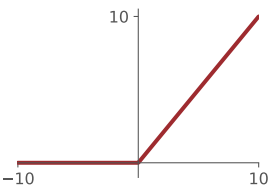
- Computes $f(x) = \max(0, x)$
- Does not saturate (in the $+$ region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)[2]
- Not zero-centered output
- An annoyance:
- Hint: what is the gradient when $x < 0$?

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks." In: *NeurIPS*. 2012

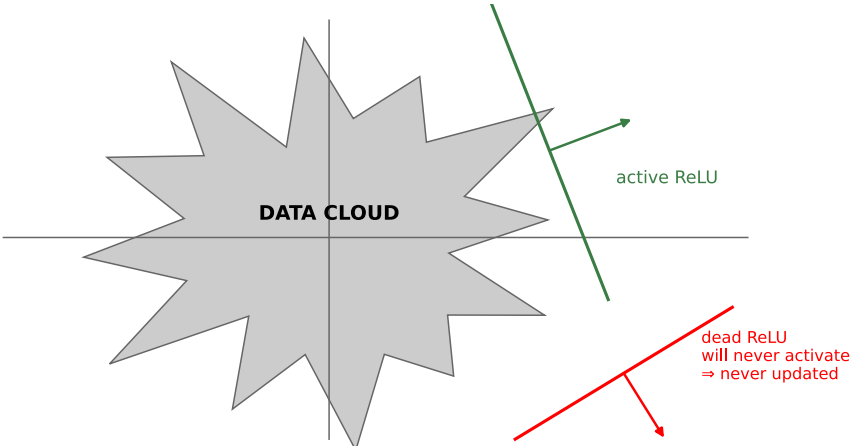
Activation Functions: ReLU



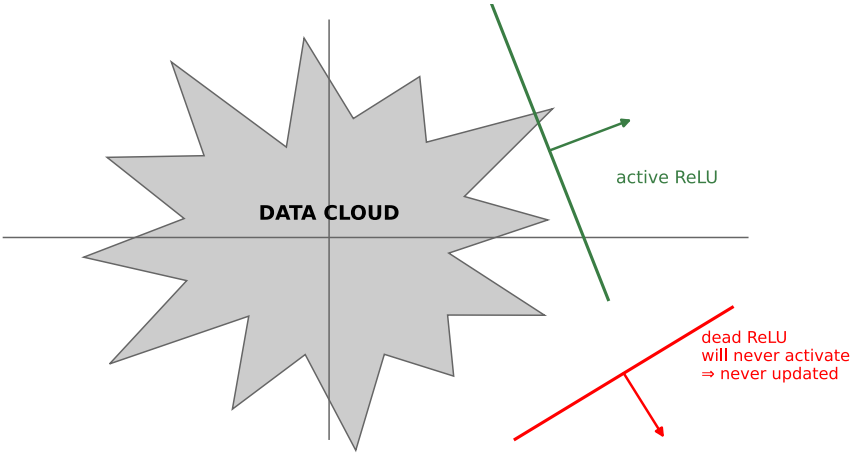
- What happens when $x = -10$?
- What happens when $x = 0$?
- What happens when $x = 10$?



Activation Functions: ReLU



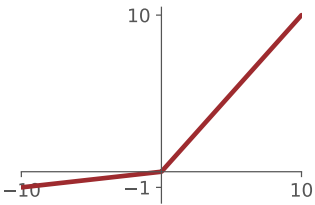
Activation Functions: ReLU



⇒ people like to initialize ReLU neurons with slightly positive biases (e.g. 0.01)

Activation Functions: Leaky ReLU

Leaky ReLU[3]



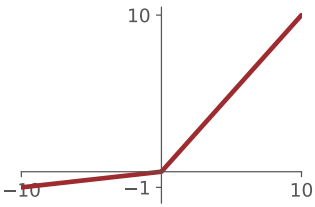
$$f(x) = \max(0.01x, x)$$

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- **Will not "die"**

[3] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. "Rectifier nonlinearities improve neural network acoustic models." In: *ICML Workshops*. 2013

Activation Functions: Leaky ReLU

Leaky ReLU[3]



$f(x) = \max(0.01x, x)$

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- **Will not "die"**

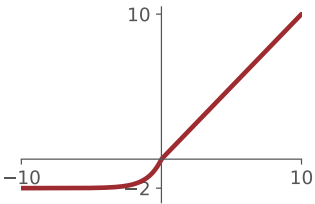
- **Parametric Rectifier (PReLU)**[4]
- $f(x) = \max(\alpha x, x)$
- backprop into α (parameter)

[3] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. "Rectifier nonlinearities improve neural network acoustic models." In: *ICML Workshops*. 2013

[4] Kaiming He et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification." In: *ICCV*. 2015

Activation Functions: ELU

Exponential Linear Units (ELU)[5]



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

- All benefits of ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise
- Computation requires exp()

[5] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. "Fast and accurate deep network learning by exponential linear units (ELUs)." In: *ICLR*. 2016

Activation Functions: Maxout



- **Maxout "neuron"**[6]
- Does not have the basic form of dot product $\rightarrow$ nonlinearity
- Generalizes ReLU and Leaky ReLU
- Linear regime! Does not saturate! Does not die!

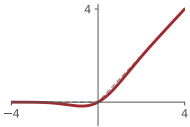
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

- Problem: doubles the number of parameters per neuron :(

[6] Ian Goodfellow et al. "Maxout networks." In: *ICML*. 2013

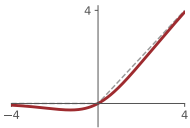
Modern Activation Functions

GELU
$x \cdot \Phi(x)$



Transformers (BERT, GPT, ViT), ConvNeXt

SiLU / Swish
$x \cdot \sigma(x)$



EfficientNet, YOLOv5 – YOLOv8, diffusion U-Nets

SwiGLU
$\mathrm{Swish}(xW) \odot (xV)$

(gated, no 1D plot)

Feed-forward blocks of LLMs (PaLM, LLaMA, Mistral, Qwen)

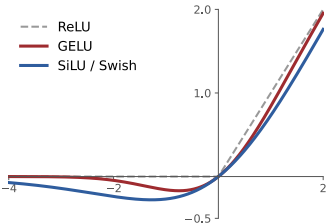
Dashed line: ReLU for comparison. Φ is the CDF of the standard normal distribution, $\odot$ is the element-wise product (explained on the SwiGLU slides).

Activation Functions: GELU

Gaussian Error Linear Unit (GELU)[7]

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

$$\approx 0.5x\left(1 + \tanh\left[\sqrt{2/\pi}\left(x + 0.044715x^3\right)\right]\right)$$



- Weights the input by the probability $\Phi(x) = P(X \leq x), X \sim \mathcal{N}(0, 1)$
- Interpretation: expected value of x multiplied by a stochastic 0/1 mask that more likely keeps larger inputs (dropout-like)
- Smooth, behaves like ReLU for large $|x|$
- Small negative values pass through – gradient is non-zero for $x < 0$
- Default in transformers (BERT, GPT, ViT) and ConvNeXt
- More expensive than ReLU

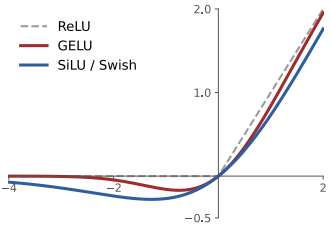
[7] Dan Hendrycks and Kevin Gimpel. “Gaussian error linear units (GELUs).” In: *arXiv* (2016)

Activation Functions: SiLU / Swish

Swish[8] / Sigmoid Linear Unit (SiLU)

$$\mathrm{Swish}_\beta(x) = x \cdot \sigma(\beta x)$$

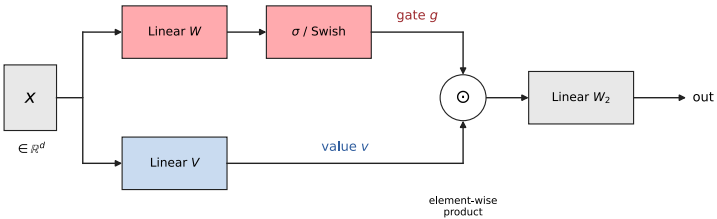
$$\mathrm{SiLU}(x) = \mathrm{Swish}_1(x) = \frac{x}{1 + e^{-x}}$$



- Found by an automated search over activation functions
- Very similar to GELU: smooth, non-monotonic, bounded below
- $\beta \to \infty$ gives ReLU, $\beta = 0$ gives $x/2$
- Often slightly better than ReLU in deep networks
- Used in EfficientNet, YOLOv5 – YOLOv8, diffusion models
- More expensive than ReLU (cheap approximation: hard-swish)

[8] Prajit Ramachandran, Barret Zoph, and Quoc V Le. "Searching for activation functions." In: *arXiv* (2017)

Activation Functions: Gated Linear Units – The Idea



- The input x is projected **twice**: a gate $g = \sigma(xW)$ and a value $v = xV$, both of size h
- $\odot$ is the **element-wise (Hadamard) product**: $(g \odot v)_k = g_k \cdot v_k$ – no summation, unlike a matrix product
- Example: $g = (0.9,\ 0.1,\ 0.5)$, $v = (2,\ -3,\ 4) \Rightarrow g \odot v = (1.8,\ -0.3,\ 2.0)$
- The gate decides **how much of each feature passes**, and since it depends on the input, the network learns which features to let through for which input (same idea as the gates in LSTMs)

Activation Functions: Gated Linear Units – Variants



- **Gated Linear Unit (GLU)**[9]: the gate uses the sigmoid, so $g_k \in (0, 1)$

$$\mathrm{GLU}(x) = \sigma(xW + b) \odot (xV + c)$$

- Variants replace σ with another activation[10]:

$$\mathrm{SwiGLU}(x) = \mathrm{Swish}(xW) \odot (xV) \qquad \mathrm{GeGLU}(x) = \mathrm{GELU}(xW) \odot (xV)$$

- Used in the feed-forward (MLP) block of transformers instead of a standard FFN:

$$\mathrm{FFN}_{\mathrm{ReLU}}(x) = \max(0, xW_1)\, W_2 \qquad \longrightarrow \qquad \mathrm{FFN}_{\mathrm{SwiGLU}}(x) = (\mathrm{Swish}(xW) \odot xV)\, W_2$$

- Consistently better than ReLU/GELU FFNs; standard in LLMs (PaLM, LLaMA, Mistral, Qwen)
- Three weight matrices instead of two (similar to Maxout) – the hidden size h is usually reduced to $\frac{2}{3}$ to keep the number of parameters the same

[9] Yann N Dauphin et al. "Language modeling with gated convolutional networks." In: *ICML*. 2017

[10] Noam Shazeer. "GLU variants improve transformer." In: *arXiv* (2020)

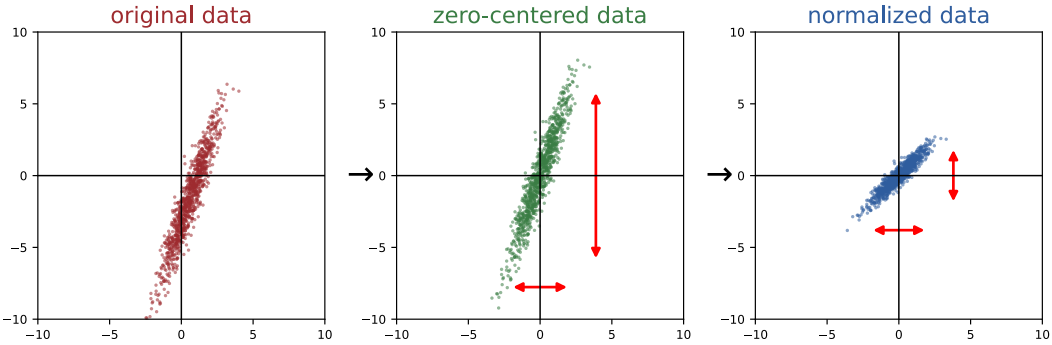
Activation Functions: Recommendations



- If you use an existing architecture, keep its activation function
- CNNs: ReLU is a solid default – simple, fast, works well. Be careful with your learning rates
- Modern CNNs and transformers: GELU or SiLU/Swish – usually a small but consistent improvement over ReLU for a bit more compute
- Transformer feed-forward blocks (LLMs): SwiGLU (or GeGLU)
- Mobile/edge devices: ReLU, ReLU6 or hard-swish (cheap, quantization-friendly)
- Leaky ReLU / ELU / Maxout: rarely needed today; try Leaky ReLU if dead ReLUs are a problem (e.g. GAN discriminators)
- tanh and sigmoid: don't use them as hidden activations – only when you need a bounded output (e.g. probabilities in $[0, 1]$, gates in LSTMs, images in $[-1, 1]$)

Data Preprocessing

Assume X of shape $[N \times D]$ is the data matrix, each example of dimension D in a row.




```
X -= np.mean(X, axis=0)
```

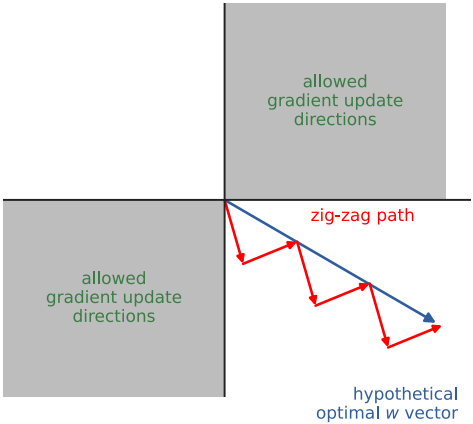
```
X /= np.std(X, axis=0)
```

Data Preprocessing

- Recall: Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

- What can we say about the gradients on W?
- They are always all positive or all negative :(

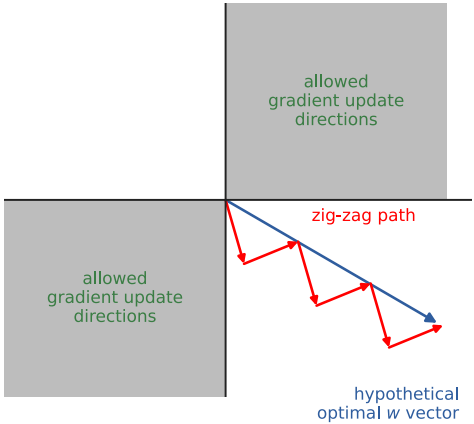


Data Preprocessing

- Recall: Consider what happens when the input to a neuron is always positive...

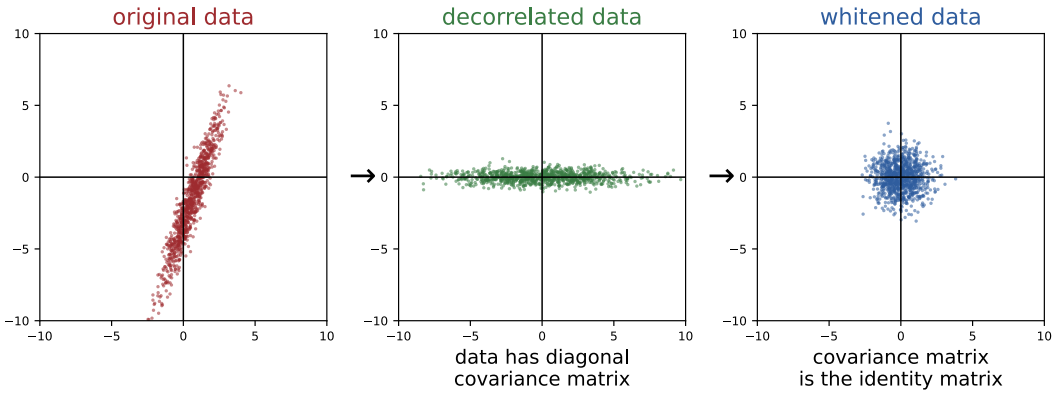
$$f\left(\sum_i w_i x_i + b\right)$$

- What can we say about the gradients on W?
- They are always all positive or all negative :(
- This is also why we want zero-centered data!



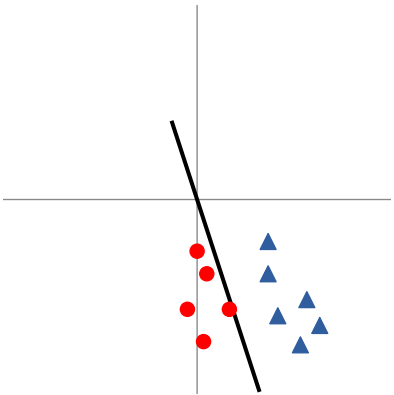
Data Preprocessing

In practice, you may also see **PCA** and **whitening** of the data:

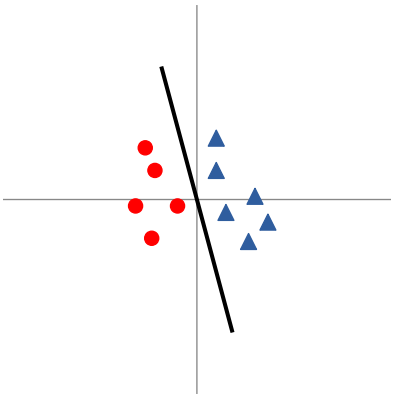


Data Preprocessing

Before normalization: classification loss very sensitive to changes in the weight matrix; hard to optimize



After normalization: less sensitive to small changes in weights; easier to optimize



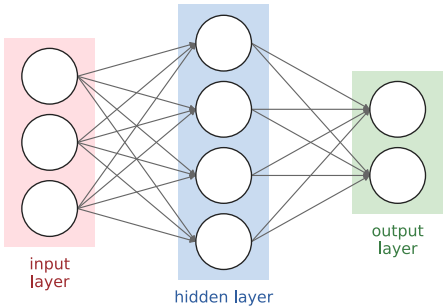
Data Preprocessing for Images: Center Only



- e.g. consider CIFAR-10 with $[32 \times 32 \times 3]$ images
- Subtract the mean image (e.g. AlexNet)
  mean image = $[32 \times 32 \times 3]$ array
- Subtract per-channel mean (e.g. VGGNet)
  mean along each channel = 3 numbers
- Subtract per-channel mean and divide by per-channel std (e.g. ResNet)
  mean and std along each channel = 3 numbers each
- Not common to do PCA or whitening

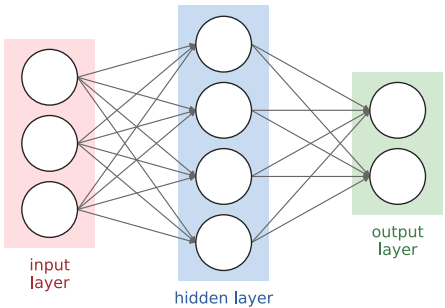
Weight Initialization

- Q: What happens if we use a constant initialization for W (e.g. $W = 0$)?



Weight Initialization

- Q: What happens if we use a constant initialization for W (e.g. $W = 0$)?



- A: All neurons in a layer compute the same output and receive the same gradient – there is no symmetry breaking!

Weight Initialization



- First idea: **Small random numbers**
- Gaussian with zero mean and standard deviation 10^{-2}

```
W = 0.01 * np.random.randn(Din, Dout)
```

- Works relatively well for small networks, but has problems with deeper networks.

Weight Initialization: Activation Statistics



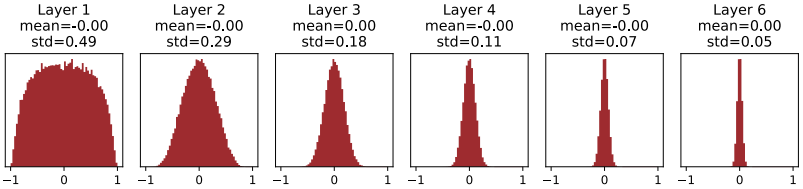
```
# 6-layer net with hidden size 4096
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Weight Initialization: Activation Statistics



```
# 6-layer net with hidden size 4096
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

- All activations tend to zero for deeper network layers
- **Q:** What do the gradients dL/dW look like?

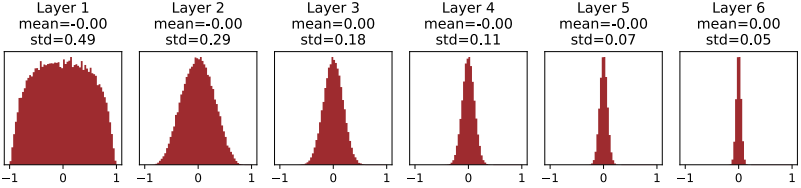


Weight Initialization: Activation Statistics



```
# 6-layer net with hidden size 4096
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

- All activations tend to zero for deeper network layers
- **Q:** What do the gradients dL/dW look like?
- **A:** All zero, no learning :(



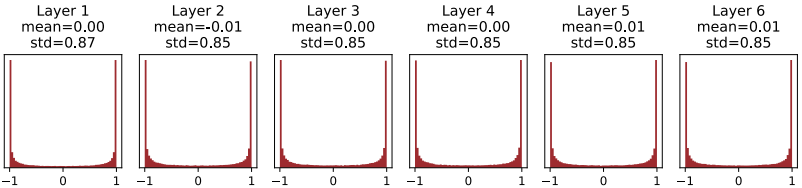
Weight Initialization: Activation Statistics



```
# Increase std of weights from 0.01 to 0.05
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Weight Initialization: Activation Statistics

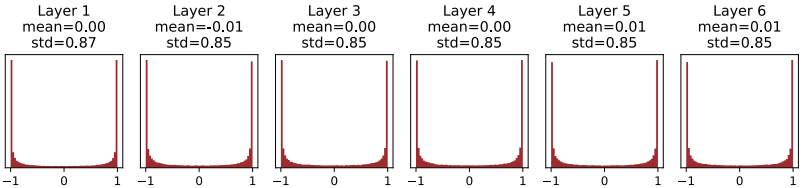
```
# Increase std of weights from 0.01 to 0.05
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```



- All activations saturate
- **Q:** What do the gradients look like?

Weight Initialization: Activation Statistics

```
# Increase std of weights from 0.01 to 0.05
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```



- All activations saturate
- **Q:** What do the gradients look like?
- **A:** Local gradients all zero, no learning :(

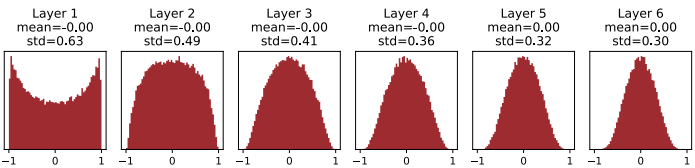
Weight Initialization: “Xavier” Initialization



```
# "Xavier" initialization: std = 1/sqrt(Din)
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Weight Initialization: “Xavier” Initialization

```
# "Xavier" initialization: std = 1/sqrt(Din)
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

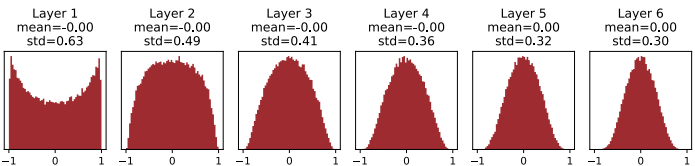


- “Just right”: Activations are nicely scaled for all layers![11]

[11] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks.” In: *AISTATS*. 2010

Weight Initialization: “Xavier” Initialization

```
# "Xavier" initialization: std = 1/sqrt(Din)
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```



- “Just right”: Activations are nicely scaled for all layers![11]
- For conv layers, D_{in} is kernel_size^2 $\cdot$ input_channels

[11] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks.” In: *AISTATS*. 2010

Weight Initialization: What about ReLU?

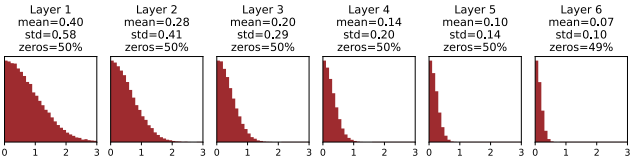


```
# Change from tanh to ReLU
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Weight Initialization: What about ReLU?

```
# Change from tanh to ReLU
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

- Xavier assumes a zero-centered activation function
- Activations collapse to zero again, no learning :(



Histograms show only the non-zero activations; about 50 % of ReLU outputs are exactly 0.

Weight Initialization: He / MSRA Initialization

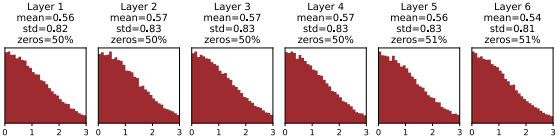


```
# ReLU correction: std = sqrt(2 / Din)
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2 / Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Weight Initialization: He / MSRA Initialization

```
# ReLU correction: std = sqrt(2 / Din)
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2 / Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

- "Just right": Activations are nicely scaled for all layers![4]
- The mean and std stay at their theoretical values ≈ 0.56 and ≈ 0.83 in every layer

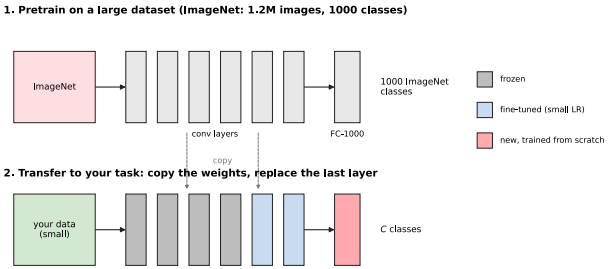


Histograms show only the non-zero activations; about 50 % of ReLU outputs are exactly 0.

[4] Kaiming He et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification." In: *ICCV*. 2015

Transfer Learning: Don’t Start from Random Weights

- Training from scratch needs a lot of labeled data – most real tasks have only little
- Better: start from a network **pretrained** on a large dataset, classically **ImageNet**[12]
- Early layers learn generic features (edges, textures), later ones task-specific[13]



[12] Jia Deng et al. “Imagenet: A large-scale hierarchical image database.” In: *CVPR*. 2009

[13] Jason Yosinski et al. “How transferable are features in deep neural networks?” In: *NeurIPS*. 2014

Transfer Learning: In Practice



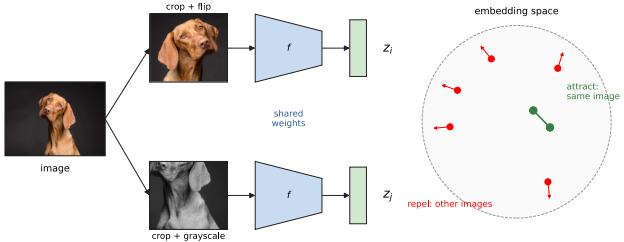
	similar dataset	**very different dataset**
small dataset	train a linear classifier on top, freeze the rest	difficult – try linear classifiers on features from different stages
medium dataset	fine-tune a few layers	fine-tune a larger number of layers (or all)
huge dataset	fine-tune all layers	fine-tune all layers

- Use a smaller learning rate for the pretrained layers (e.g. $\frac{1}{10}$ of the original)
- Use the **same preprocessing** as during pretraining (e.g. ImageNet per-channel mean and std)
- Pretraining is the norm, not the exception – random init is mostly used when training on huge datasets or with new architectures

```
model = torchvision.models.resnet50(weights="IMAGENET1K_V2")  # pretrained weights
model.fc = torch.nn.Linear(2048, num_classes)                 # new last layer
```

Self-Supervised Pretraining: Contrastive Learning

- Labels are expensive, unlabeled images are cheap ⇒ **self-supervised learning**: create the training signal from the data itself, then transfer the network as before
- **Contrastive learning** (e.g. SimCLR[14]): two augmentations of the same image → similar embeddings, different images → dissimilar embeddings

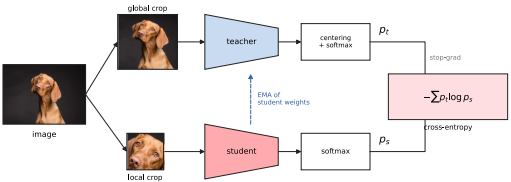


$$\ell_{i,j} = -\log \frac{\exp\left(\mathrm{sim}(z_i, z_j)/\tau\right)}{\sum_{k \neq i} \exp\left(\mathrm{sim}(z_i, z_k)/\tau\right)}$$

sim = cosine similarity, τ = temperature

[14] Ting Chen et al. "A simple framework for contrastive learning of visual representations." In: *ICML*. 2020

Self-Supervised Pretraining: DINO



- **Self-distillation with no labels**[15]: the student learns to predict the teacher's output for a different crop of the same image
- Teacher = exponential moving average (EMA) of the student; it sees only global crops, the student also local ones $\Rightarrow$ learns local-to-global correspondences
- No negative pairs needed; collapse is avoided by centering and sharpening the teacher's outputs
- DINOv2[16]: strong general-purpose features, often used frozen with just a linear head

[15] Mathilde Caron et al. "Emerging properties in self-supervised vision transformers." In: *ICCV*. 2021

[16] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. "DINOv2: Learning robust visual features without supervision." In: *TMLR* (2024)

Batch Normalization

- Batch normalization (BN)[17]
- "You want zero-mean unit-variance activations? Just make them so."
- Consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply:

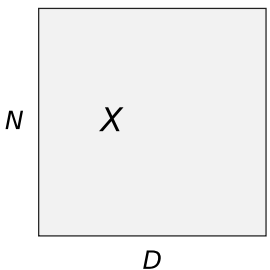
$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}\left[x^{(k)}\right]}{\sqrt{\mathrm{Var}\left[x^{(k)}\right]}}$$

This is a vanilla differentiable function...

[17] Sergey Ioffe and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." In: *ICML*. 2015

Batch Normalization

Input: $x : N \times D$



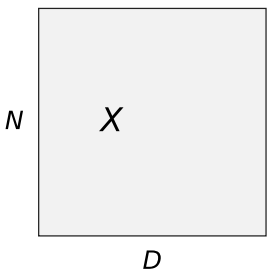
$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel variance, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, shape is $N \times D$

Batch Normalization

Input: $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel variance, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, shape is $N \times D$

Problem: What if zero-mean, unit variance is too hard of a constraint?

Batch Normalization



Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function (= no BN)

$$\mu_j = \frac{1}{N} \sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^{N} (x_{i,j} - \mu_j)^2$$ Per-channel variance, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, shape is $N \times D$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, shape is $N \times D$

Batch Normalization: Test-Time



Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function (= no BN)

Estimates depend on minibatch; can't do this at test-time!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel variance, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, shape is $N \times D$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, shape is $N \times D$

Batch Normalization: Test-Time



Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function (= no BN)

$\mu_j =$ (Running) average of values seen during training — Per-channel mean, shape is D

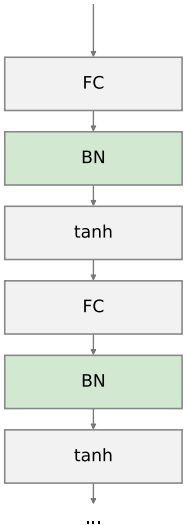
$\sigma_j^2 =$ (Running) average of values seen during training — Per-channel variance, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ — Normalized x, shape is $N \times D$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ — Output, shape is $N \times D$

During testing BN becomes a linear operator! It can be fused with the previous fully-connected or conv layer.

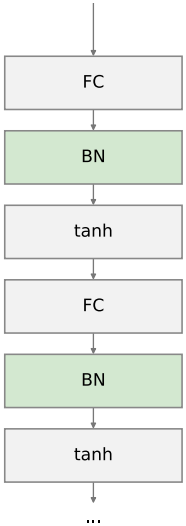
Batch Normalization



Usually inserted after fully connected or convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}\left[x^{(k)}\right]}{\sqrt{\mathrm{Var}\left[x^{(k)}\right]}}$$

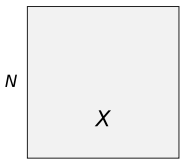
Batch Normalization



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!
- Does not work well with small batches

Layer Normalization

Input: $x : N \times D$



$$\mu_i = \frac{1}{D}\sum_{j=1}^{D} x_{i,j}$$ Per-sample mean, shape is N

$$\sigma_i^2 = \frac{1}{D}\sum_{j=1}^{D}(x_{i,j} - \mu_i)^2$$ Per-sample variance, shape is N

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$ Normalized x, shape is $N \times D$

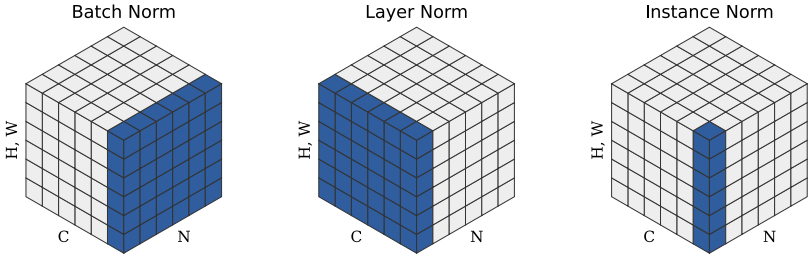
$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, shape is $N \times D$

- Layer norm[18]: same as BN, but the statistics are computed **over the features of each sample** instead of over the batch
- Independent of the batch size, identical behavior during training and testing
- Standard in transformers and RNNs (also ConvNeXt)
- RMSNorm[19] (used in LLMs) skips the mean: $\hat{x}_{i,j} = x_{i,j} / \sqrt{\frac{1}{D}\sum_j x_{i,j}^2 + \varepsilon}$

[18] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer normalization." In: *arXiv* (2016)

[19] Biao Zhang and Rico Sennrich. "Root mean square layer normalization." In: *NeurIPS*. 2019

Comparison of Normalization Layers



17 18 20

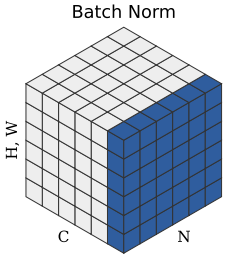
[17] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift.” In: *ICML*. 2015

[18] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer normalization.” In: *arXiv* (2016)

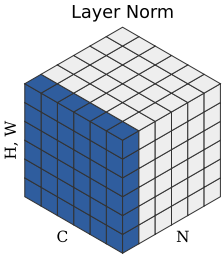
[20] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance normalization: The missing ingredient for fast stylization.” In: *arXiv* (2016)

[21] Yuxin Wu and Kaiming He. “Group normalization.” In: *ECCV*. 2018

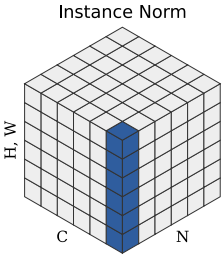
Comparison of Normalization Layers



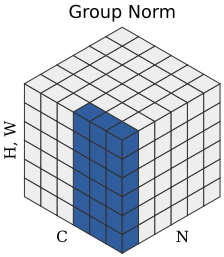
17



18



20



21

[17] Sergey Ioffe and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." In: *ICML*. 2015

[18] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer normalization." In: *arXiv* (2016)

[20] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. "Instance normalization: The missing ingredient for fast stylization." In: *arXiv* (2016)

[21] Yuxin Wu and Kaiming He. "Group normalization." In: *ECCV*. 2018

Summary



We looked in detail at:

- Activation functions (ReLU for CNNs, GELU/SiLU for modern nets, SwiGLU in LLMs)
- Data preprocessing (images: subtract mean, divide by std)
- Weight initialization (Xavier/He when training from scratch)
- Transfer learning (start from pretrained weights: ImageNet or self-supervised, e.g. DINOv2)
- Normalization (BatchNorm in CNNs, LayerNorm/RMSNorm in transformers)